```
  private def system(*args)
    super.tap do |result|
      puts "system(#{args.join(', ')}) returned #{result.inspect}"
    end
  end
end

class Object
  prepend SystemHook
end

system("date")
system("kangaroo", "-hop 10", "skippy")
```

produces:

```
Sun Dec 25 12:30:34 CST 2022
system(date) returned true
system(kangaroo, -hop 10, skippy) returned nil
```

Object Creation Hooks

Ruby lets you get involved when objects are created. If you can be present when every object is born, you can do all sorts of interesting things: you can wrap them, add methods to them, remove methods from them, and add them to containers to implement persistence—you name it. We'll show a simple example here. We'll add a timestamp to every object as it's created. First, we'll add a `timestamp` attribute to every object in the system. We can do this by hacking class `Object` itself:

```
class Object
  attr_accessor :timestamp
end
```

Then, we need to hook object creation to add this timestamp. One way to do this is to do our method-renaming trick on `new`—the method that's called to allocate space for a new object. The technique isn't perfect—some built-in objects, such as literal strings, are constructed without calling `new`—but it'll work just fine for objects we write:

```
class Class
  old_new = instance_method(:new)
  define_method(:new) do |*args, **kwargs, &block|
    result = old_new.bind_call(self, *args, **kwargs, &block)
    result.timestamp = Time.now unless result.is_a?(Time)
    result
  end
end
```

Finally, we can run a test. We'll create a couple of objects a few milliseconds apart and check their timestamps:

```
class Test
end

obj1 = Test.new
sleep(0.002)
obj2 = Test.new
obj1.timestamp.to_f # => 1671993034.184892
obj2.timestamp.to_f # => 1671993034.1871612
```